



Formação Backend: Python & FastAPI — O Caminho Sannin de Orochimaru

Projeto Mushi Bingo

Aula Anterior

- Definição de back-end
- Definição de API
- Definição do protocolo HTTP
 - Verbos HTTP
 - Status Code
- Definição de Endpoints
- Ambiente de Desenvolver
- CRUD



Boas práticas em Python

Estilo e legibilidade

- Indentação com 4 espaços
- Linha deve ter no máximo 79 caracteres
- Declarações
 - variáveis e funções em snake_case
 - classes em PascalCase
 - constantes em UPPER_SNAKE

Dica: Utilize ferramentas com `ruff` para garantir as boas práticas em Python

Banco de Dados (Revisão) - SGBD

É um conjunto organizado de dados relacionados, armazenado e gerenciado de forma que permita inserção, consulta, atualização e remoção eficientes.

Modelagem de Dados:

- conceitual
- lógico
- físico

Normalização:

- 1FN
- 2FN
- 3FN

Modelagem de Dados

Conceitual

Representação de alto nível dos dados e suas relações, independente de tecnologia.

- Objetivo: capturar entidades, atributos e relacionamentos entendidos pelos stakeholders.
- Independente do SGBD.

```
cliente:  
pedido
```

```
pedido:  
produto
```

Lógico

Mapeamento do modelo conceitual para estruturas lógicas (tabelas, colunas, chaves).



- Objetivo: definir chaves primárias/estrangeiras, cardinalidades e tipos de dados abstratos.
- Independente do SGBD.

Físico

Implementação concreta no SGBD escolhido, com decisões de armazenamento, índices e tipos concretos.



- Objetivo: otimizar desempenho, armazenamento e integridade no ambiente de produção
- Dependente do SGBD.

Normalização

Primeira Forma Normal (1FN)

Cada campo deve conter valores atômicos; eliminar grupos repetitivos/colunas multivaloradas.

- Objetivo: facilita consultas e evita redundância de dados dentro de uma tupla.



Segunda Forma Normal (2FN)

Estar em 1FN e todos os atributos não-chave dependerem funcionalmente da chave primária inteira (eliminar dependências parciais).

- Objetivo: evitar duplicação quando a chave primária é composta.



Terceira Forma Normal (3FN)

Estar em 2FN e nenhum atributo não-chave depender de outro atributo não-chave (eliminar dependências transitivas).

- Objetivo: reduzir anomalias de atualização e mantém integridade sem redundância desnecessária.



Comandos básicos de SQL

- criar

```
CREATE TABLE clientes (  
    id SERIAL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(255) UNIQUE  
);
```

Comandos básicos de SQL

- listar

```
SELECT id, nome, email  
FROM clientes  
WHERE nome = 'João Silva';
```

Comandos básicos de SQL

- delatar

```
DELETE FROM clientes  
WHERE id = 1;
```

Comandos básicos de SQL

- inserir

```
INSERT INTO clientes (nome, email)  
VALUES ('João Silva', 'joao@email.com');
```

Comandos básicos de SQL

- atualizar

```
UPDATE clientes  
SET email = 'novoemail@email.com'  
WHERE id = 1;
```

Boas práticas para banco de dados

Nomenclatura: Regras Gerais

- Padrão Snake Case:
 - *Correto:* `data_nascimento`
 - *Incorreto:* `DataNascimento`, `dataNascimento`
- Idioma Unificado
- Caracteres Permitidos
- Palavras Reservadas

Nomenclatura: Tabelas

- Sempre no Singular:
 - *Correto:* cliente, item_pedido
- Evite prefixos como tb_ ou tbl_.
 - *Correto:* usuario
 - *Incorreto:* tb_usuario

Nomenclatura: Atributos (Colunas)

- Sempre no Singular:
 - *Correto:* nome, preco_unitario
- Evite Redundância
 - *Correto:* nome, cpf
 - *Incorreto:* nome_cliente, cpf_cliente
- Seja Descritivo e Claro
 - *Correto:* quantidade_estoque
 - *Incorreto:* qtd_est

Nomenclatura: Chaves (PK e FK)

- Chave Primária (PK): Utilizar simplesmente `id`.
- Chave Estrangeira (FK): Deve seguir a regra estrita de usar o nome da tabela referenciada no singular, seguido do sufixo `_id`.
 - *Exemplo em uma tabela de pedidos:* `cliente_id`, `produto_id`.

O projeto Weekly

Objetivo: Automatizar e organizar como Weekly atuais são feitas. Por meio de uma API, é possível integrar ao discord e outros sistemas.

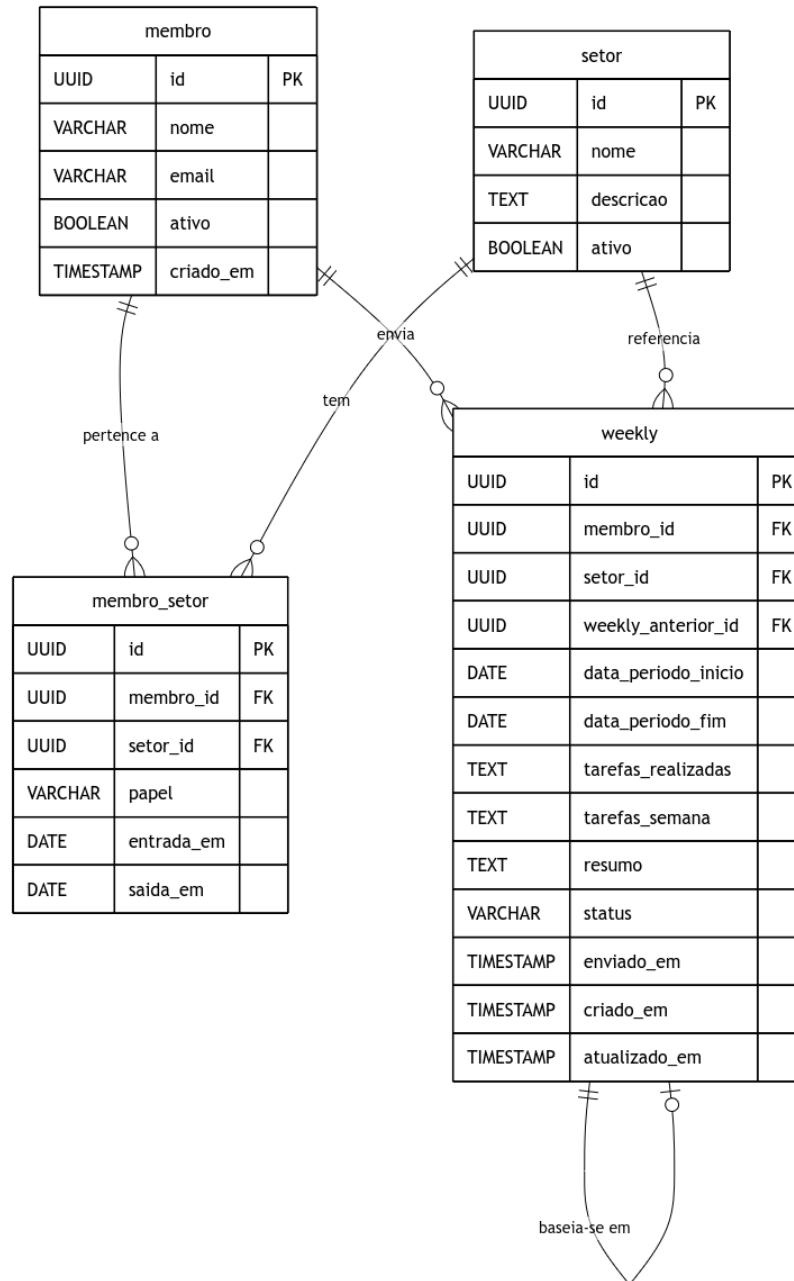


Diagrama RE

É uma representação visual que descreve a estrutura lógica de um banco de dados, facilitando a compreensão das regras de negócio pela equipe.

Componentes fundamentais:

- Entidades (tabelas)
- Atributos (colunas)
- Relacionamentos



Regras de negócio: Fluxo de Status

Fluxo de aprovação:

- `rascunho` -> `enviado` -> `aprovado`
 - *Regra:* Somente segunda-feira para criação e envio.

Fluxo de expiração:

- `rascunho` -> `expirado`
 - *Regra:* Automático na virada para terça-feira.

Regras de negócio: Membros e Setores

- Um membro pode pertencer a vários setores simultaneamente.
- O papel (`membro`, `vice_lider`, `lider`) é definido por setor.
 - *Exemplo:* O usuário pode ser líder no setor A e membro comum no setor B.
- Desativar um membro ou setor não apaga o histórico de weeklies.
- O envio de weekly é permitido apenas para setores onde o membro está ativo (`saida_em IS NULL`).

Regras de negócio: Janela de envio

- Período de ação: Criação, edição e envio do weekly ocorrem **somente na segunda-feira**.
- Datas do período: `data_periodo_inicio` deve ser uma segunda-feira; `data_periodo_fim` a sexta-feira da mesma semana.
- Expiração: Rascunhos não enviados até a virada para terça-feira são marcados como `expirado` automaticamente pelo sistema.

Regras de negócio: Weekly

- Limite de **um weekly** por membro, por setor, por semana. Duplicatas são rejeitadas.
- O campo `weekly_anterior_id` deve apontar para o weekly da semana imediatamente anterior (aceita nulo na primeira semana).
- Weeklies com status `aprovado` ou `expirado` **não podem** ser editados ou deletados.
- Apenas weeklies com status `enviado` podem ser aprovados.

Regras de negócio: Relatório de frequência

- Disponível para consulta somente a partir de **terça-feira**.
- Acesso restrito a usuários com papel de `lider` ou `vice_lider` no setor consultado.
- Lista todos os membros ativos do setor com os status: `enviado`, `aprovado` ou `não enviou`.
- O parâmetro `?semana=` permite consultar dados de semanas anteriores sem restrição de dia da semana.

Endpoints (Sugestão): Membros

- `GET /membros` : Lista membros (Filtro: `?ativo=`).
- `GET /membros/{id}` : Retorna membro com setores ativos.
- `POST /membros` : Cria um novo membro.
- `PATCH /membros/{id}` : Atualiza dados ou desativa o membro.
- `GET /membros/{id}/setores` : Lista setores do membro com datas e papel.

Endpoints (Sugestão): Setores

- `GET /setores` : Lista setores.
- `GET /setores/{id}` : Retorna setor detalhado com membros ativos.
- `POST /setores` : Cria um novo setor.
- `PATCH /setores/{id}` : Atualiza ou desativa o setor.
- `POST /setores/{id}/membros` : Adiciona membro ao setor definindo o seu papel.
- `PATCH /setores/{id}/membros/{membro_id}` : Remove membro do setor (preenche o campo `saida_em`).

Endpoints (Sugestão): Weeklies

- `GET /weeklies` : Lista weeklies (Filtros: `?membro_id=`, `?setor_id=`, `?status=`, `?semana=`).
- `GET /weeklies/{id}` : Retorna weekly com os dados do anterior linkado.
- `POST /weeklies` : Cria rascunho (*somente na segunda-feira*).
- `PATCH /weeklies/{id}` : Editar rascunho (*bloqueado se aprovado/expirado*).
- `POST /weeklies/{id}/enviar` : Transição de rascunho para enviado.
- `POST /weeklies/{id}/aprovar` : Transição para aprovado (*restrito a líder/vice; somente terça+*).
- `DELETE /weeklies/{id}` : Remove weekly (*permitido apenas para rascunhos*).

Endpoints (Sugestão): Relatórios

- `GET /setores/{id}/relatorio-frequencia` : Frequência de envio da semana.
 - *Regras aplicadas:* Requer papel de líder ou vice; acesso liberado a partir de terça-feira; aceita parâmetro opcional `?semana=`.
- `GET /membros/{id}/historico` : Retorna a cadeia cronológica de weeklies de um membro específico.

Estrutura de Pasta (Projeto)

```
weekly
├── app
│   ├── main.py
│   ├── models
│   ├── routers
│   ├── schema
│   └── settings.py
├── test
├── migrations
├── database.db
├── alembic.ini
└── .env
```

ORM (Object-Relational Mapping)

É uma técnica de programação (ferramenta) que permite interagir diretamente com um banco de dados usando o paradigma de Orientação a Objetos da sua linguagem de programação, em vez de escrever consultas SQL puras.

- Abstração de banco de dados
- Segurança
- Eficiência no desenvolvimento

SQLAlchemy

É o ORM mais robusto e tradicional do ecossistema Python. Ele abstrai a complexidade do banco de dados relacional, permitindo que você manipule tabelas e registros como se fossem classes e objetos Python nativos.

- Mapeamento Objeto-Relacional
- Abstração de Dialeto
- Gerenciamento de Sessão

Para instalar

```
# No linux e Windows  
uv add sqlalchemy
```

Migrations

É o conceito de controle de versão aplicado ao esquema (estrutura) do banco de dados. As migrações registram cada evolução da estrutura em arquivos de código ordenados cronologicamente.

- Histórico de Evolução
- Consistência de Ambientes
- Preservação de Dados

Alembic

É a ferramenta oficial de migração de banco de dados projetada especificamente para funcionar em conjunto com o SQLAlchemy. Ele automatiza o processo de criação e aplicação das migrações.

- Autogeração de scripts
- Controle de direção (Upgrade/Downgrade)
- Automação de deploy

Para instalar

```
# No linux e Windows
uv add alembic

uv run alembic init migrations

uv run alembic revision --autogenerate -m "mensagem"
```



Atividade

- Leia as regras de negócios e tente implementar os recursos (endpoints)
- Tente criar por si mesmo as tabelas do banco de dados
- Tente refazer o diagrama do banco de dados conforme seu gosto (assim como as regras de negócio e endpoints)

Próximos tópicos

- Conectar Banco de Dados
- Router
- Criar Recursos
- RedirectResponse
- Injeção de Dependência
- Middleware

Referencias

- FastAPI do Zero
- FastAPI
- UV
- Pydantic
- Métodos de requisição HTTP
- REST: Princípios e boas práticas
- 5 dicas para fazer APIs melhores
- Alembic
- SQLAlchemy
- Ambiente Python Moderno 2025: UV, Ruff, Pyright, pyproject.toml e VS Code
- Curso de Modelagem de Dados



Até a próxima!